GROT Yanis GROUPE 3

TPR201 : Routage dynamique avec RIP

Sommaire :

- Introduction

- Exo 1 : Mise en place du réseau et configuration des sous-réseaux locaux
- Exo 2 : Routes statiques avec Zebra

- Exo 3 : Routage dynamique avec RIP

- Exo 4 : Ajout d’un routeur à chaud

- Exo 5 : Tolérance aux pannes et résilience

Introduction :

Ce TP est consacré à l'implémentation du routage dynamique via le protocole RIP (*Routing Information Protocol*). Contrairement au routage statique, cette méthode permet aux routeurs de construire automatiquement leurs tables de routage et de s'adapter en temps réel aux évolutions de la topologie sans intervention humaine. L'objectif principal est de configurer les interfaces réseau, d'activer les démons de la suite Quagga (Zebra pour la gestion des routes et ripd pour le protocole RIP) et d'analyser la propagation des informations entre les équipements. Une phase de test dédiée à la résilience permettra également de vérifier la capacité du réseau à maintenir la connectivité lors de la simulation d'une panne.

Le cadre technique repose sur le simulateur Marionnet, exploitant des machines virtuelles sous Debian. La suite logicielle open-source Quagga est utilisée pour piloter les fonctions de routage. Quant à l'architecture réseau, elle s'articule autour de trois routeurs (R1, R2 et R3) interconnectés en série par des liaisons directes. Chacun de ces routeurs dessert des machines clientes (nommées m1 à m4), lesquelles sont rattachées à leur passerelle respective par l'intermédiaire de quatre commutateurs distincts.

Exo 1 : Mise en place du réseau et configuration des sous-réseaux locaux

1. Configurez les interfaces réseaux des machines et leur passerelle par défaut.

Pour m1 :

```
[0 root@m1 ~]$ ifconfig eth0 192.168.10.1/24 up
[0 root@m1 ~]$ route add default gw 192.168.10.254
[0 root@m1 ~]$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 02:04:06:7c:f3:66
          inet addr:192.168.10.1  Bcast:192.168.10.255  Mask:255.255.255.0
          inet6 addr: fe80::4:6ff:fe7c:f366/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:7 errors:0 dropped:0 overruns:0 frame:0
          TX packets:7 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:444 (444.0 B)  TX bytes:500 (500.0 B)
          Interrupt:5

[0 root@m1 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         192.168.10.254  0.0.0.0         UG    0      0        0 eth0
192.168.10.0    *               255.255.255.0   U     0      0        0 eth0
[0 root@m1 ~]$
```

Pour m2 :

```
[0 root@m2 ~]$ ifconfig eth0 192.168.20.1/24 up
[0 root@m2 ~]$ route add default gw 192.168.20.254
[0 root@m2 ~]$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 02:04:06:01:8d:fd
          inet addr:192.168.20.1  Bcast:192.168.20.255  Mask:255.255.255.0
          inet6 addr: fe80::4:6ff:fe01:8dfd/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:6 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:468 (468.0 B)
          Interrupt:5

[0 root@m2 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         192.168.20.254  0.0.0.0         UG    0      0        0 eth0
192.168.20.0    *               255.255.255.0   U     0      0        0 eth0
[0 root@m2 ~]$
```

Pour m3 :

```
[0 root@m3 ~]$ ifconfig eth0 192.168.30.1/24 up
[0 root@m3 ~]$ route add default gw 192.168.30.254
[0 root@m3 ~]$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 02:04:06:6c:69:83
          inet addr:192.168.30.1  Bcast:192.168.30.255  Mask:255.255.255.0
          inet6 addr: fe80::4:6ff:fe6c:6983/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:10 errors:0 dropped:0 overruns:0 frame:0
          TX packets:6 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:498 (498.0 B)  TX bytes:468 (468.0 B)
          Interrupt:5

[0 root@m3 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         192.168.30.254  0.0.0.0         UG    0      0        0 eth0
192.168.30.0    *               255.255.255.0   U     0      0        0 eth0
[0 root@m3 ~]$
```

Pour m4 :

```
[0 root@m4 ~]$ ifconfig eth0 192.168.40.1/24 up
[0 root@m4 ~]$ route add default gw 192.168.40.254
[0 root@m4 ~]$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 02:04:06:17:be:4a
          inet addr:192.168.40.1  Bcast:192.168.40.255  Mask:255.255.255.0
          inet6 addr: fe80::4:6ff:fe17:be4a/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:12 errors:0 dropped:0 overruns:0 frame:0
          TX packets:7 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:534 (534.0 B)  TX bytes:500 (500.0 B)
          Interrupt:5

[0 root@m4 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         192.168.40.254  0.0.0.0         UG    0      0        0 eth0
192.168.40.0    *               255.255.255.0   U     0      0        0 eth0
[0 root@m4 ~]$
```

2. Configurez les interfaces réseaux des routeurs. Ne touchez pas à leurs tables de routage !

Pour R1 :

```
[0 root@R1 ~]$ ifconfig eth0 192.168.10.254/24 up
[0 root@R1 ~]$ ifconfig eth1 192.168.20.254/24 up
[0 root@R1 ~]$ ifconfig eth2 192.168.1.1/24 up
[0 root@R1 ~]$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.10.254  netmask 255.255.255.0  broadcast 192.168.10.255
        inet6 fe80::4:6ff:fe62:8ecc  prefixlen 64  scopeid 0x20<link>
        ether 02:04:06:62:8e:cc  txqueuelen 1000  (Ethernet)
        RX packets 18  bytes 600 (600.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 18  bytes 894 (894.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 5

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.20.254  netmask 255.255.255.0  broadcast 192.168.20.255
        inet6 fe80::4:6ff:fe5f:85b1  prefixlen 64  scopeid 0x20<link>
        ether 02:04:06:5f:85:b1  txqueuelen 1000  (Ethernet)
        RX packets 15  bytes 546 (546.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 5  bytes 440 (440.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 5

eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.1  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::4:6ff:fe74:ef0b  prefixlen 64  scopeid 0x20<link>
        ether 02:04:06:74:ef:0b  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 4  bytes 408 (408.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 5
```

Pour R2 :

```
[0 root@R2 ~]$ ifconfig eth0 192.168.30.254/24 up
[0 root@R2 ~]$ ifconfig eth1 192.168.1.2/24 up
[0 root@R2 ~]$ ifconfig eth2 192.168.2.1/24 up
[0 root@R2 ~]$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.30.254  netmask 255.255.255.0  broadcast 192.168.30.255
        inet6 fe80::4:6ff:feca:742a  prefixlen 64  scopeid 0x20<link>
        ether 02:04:06:ca:74:2a  txqueuelen 1000  (Ethernet)
        RX packets 15  bytes 546 (546.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 19  bytes 926 (926.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 5

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.2  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::4:6ff:feee:3599  prefixlen 64  scopeid 0x20<link>
        ether 02:04:06:ee:35:99  txqueuelen 1000  (Ethernet)
        RX packets 5  bytes 370 (370.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 5  bytes 478 (478.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 5

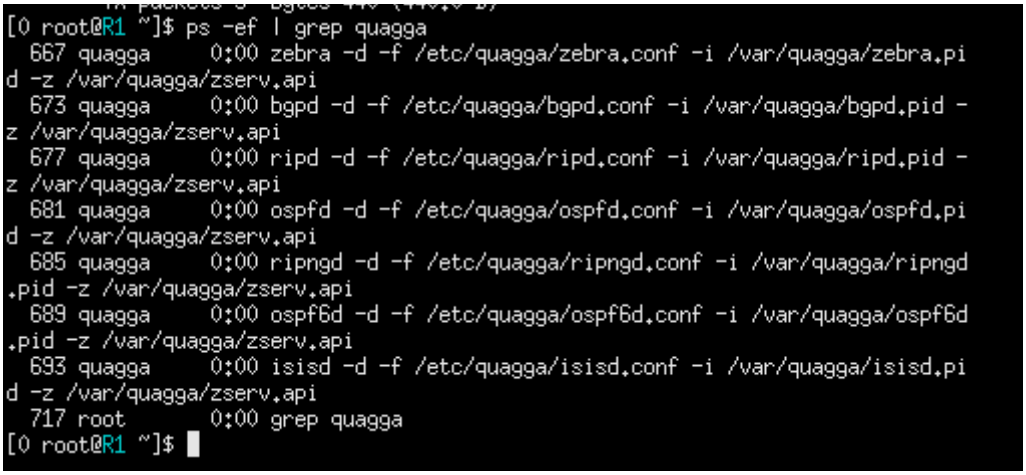
eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.2.1  netmask 255.255.255.0  broadcast 192.168.2.255
        inet6 fe80::4:6ff:fe49:eb53  prefixlen 64  scopeid 0x20<link>
        ether 02:04:06:49:eb:53  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 3  bytes 318 (318.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 5
```

3. Avec la commande ps -ef | grep quagga, observez quels programmes tournent pour Quagga.

La configuration de chaque démon en charge d'un protocole est située dans un fichier spécifique qui porte son nom situé dans le répertoire /etc/quagga. Quels sont les fichiers qui vont nous intéresser ici ?

On entre cette commande sur chaque routeur :

Exemple sur R1 :



Les fichiers qui vont nous intéresser sont les fichiers :
- zebra.conf
- ripd.conf

Dans ce premier exo, nous avons bien configuré toutes les interfaces des machines grâce à la commande ifconfig et nous avons aussi configuré les passerelles par défauts. Les interfaces des routeurs sont aussi bien configurées et nous allons maintenant passer à la configuration pour le protocole RIP.

Exo 2 : Routes statiques avec Zebra

1. Vous avez déterminé dans la partie précédente dans quel fichier est configuré le démon en charge de Zebra. Éditez ce fichier : il contient un exemple de fichier de configuration.

Modifiez-le pour utiliser le vrai hostname des routeurs. Effectuez cette opération sur les trois routeurs.

Pour R1 :

```
!---
! ZEBRA configuration file (Quagga port 2601)
!---
! IP routing manager
! See: http://www.nongnu.org/quagga/docs/quagga.html
!---
! Note: leave passwords unchanged or change them in the same way for
! all selected services (zebra, rip, ripng, ospf, bgp, ospf6, isis)
password zebra
enable password zebra
!log file /var/quagga/zebra.log
!
!=== INTERFACE CONFIGURATION ===
!interface IFNAME
!  ip   address ADDRESS/PREFIX
!  ipv6 address ADDRESS/PREFIX
!  ..
!exit
!=== STATIC ROUTING ===
!ip   forwarding
!ipv6 forwarding
!ip   route ADDRESS/PREFIX GATEWAY
!ipv6 route ADDRESS/PREFIX GATEWAY
!
!=== EXAMPLES (tip: uncomment and adapt) ===
!interface eth1
!  ip   address 10.10.0.5/16
!  ipv6 address 2001:db9::5/64
!exit
!ip   route 11.11.11.0/24 10.10.255.254
!ipv6 route 2001:db8::/64 2001:db9::ff
!---

hostname R1
```

Il n’y avait pas de ligne hostame, je l’ai donc rajouté en fin de fichier. Je fais les mêmes étapes avec R2 et R3 avec leurs hostname respectifs.

Je redémarre le service :

```
[0 root@R1 ~]$ /etc/init.d/quagga stop
[1 root@R1 ~]$ /etc/init.d/quagga start
[0 root@R1 ~]$
```

2. Pour configurer le démon Zebra, on s'y connecte avec l'utilitaire Telnet. Il écoute sur le port 2601. Utilisez le mot de passe que vous avez défini dans le fichier de configuration.

La commande show ip route permet d'afficher les routes statiques reconnues par Zebra. Ne tenez pas compte de l'interface eth42 et du réseau 172.23.0.0/16 : il s'agit d'une interface utilisée en interne par Marionnet. Sur chacun de vos trois routeurs, quelles sont les routes vues par Zebra ?

Sur R1 :

```
R1# show ip route
Codes: K - kernel route, C - connected, S - static, R - RIP,
       O - OSPF, I - IS-IS, B - BGP, A - Babel,
       > - selected route, * - FIB route

C>* 127.0.0.0/8 is directly connected, lo
C>* 192.168.1.0/24 is directly connected, eth2
C>* 192.168.10.0/24 is directly connected, eth0
C>* 192.168.20.0/24 is directly connected, eth1
R1#
```

Zebra connaît les routes sur R1 :

- 192.168.1.0/24 eth2
- 192.168.10.0/24 eth0
- 192.168.20.0/24 eth1

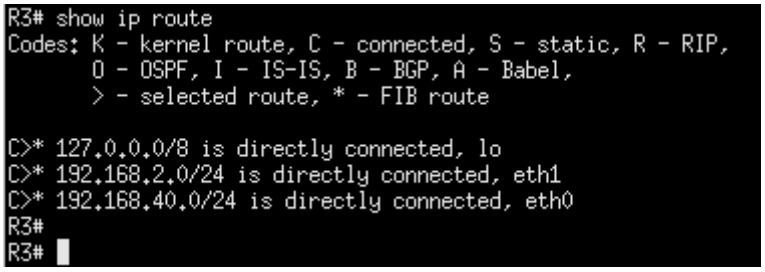
On fait la même chose sur les autres routeurs.

Sur R2 :

```
R2# show ip route
Codes: K - kernel route, C - connected, S - static, R - RIP,
       O - OSPF, I - IS-IS, B - BGP, A - Babel,
       > - selected route, * - FIB route

C>* 127.0.0.0/8 is directly connected, lo
C>* 192.168.1.0/24 is directly connected, eth1
C>* 192.168.2.0/24 is directly connected, eth2
C>* 192.168.30.0/24 is directly connected, eth0
R2#
R2#
```

Sur R3 :

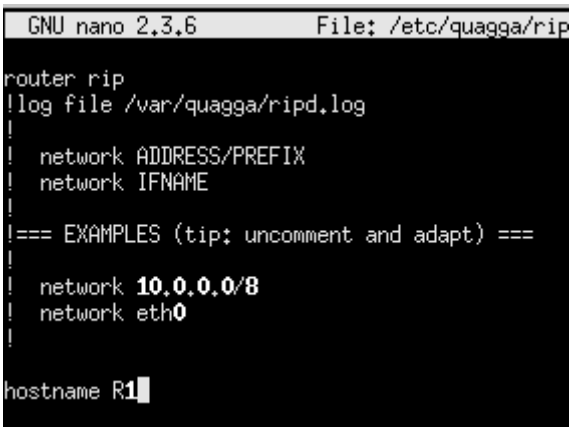


Nous avons ici vérifier les routes auxquelles zebra a accès, après cette vérification, on peut passer à la suite du TP.

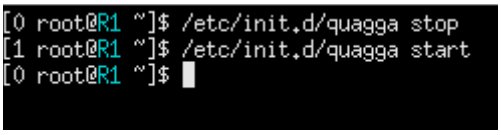
Exo 3 : Routage dynamique avec RIP

1. De la même manière que pour configurer Zebra, éditez le fichier de configuration de RIP afin d'attribuer le bon hostname sur chacun de vos routeurs. N'oubliez pas de redémarrer Quagga après votre modification. Effectuez cette opération sur les trois routeurs.

Pour R1 :



Et on redémarre le service :



On fait la même chose sur R2 et R3 :



Pour R3 :



Pour configurer le démon RIP, on s'y connecte également avec l'utilitaire telnet, cette fois sur le porte 2602. Utilisez le mot de passe que vous avez défini dans le fichier de configuration.

3. À quels réseaux est relié directement chacun de vos routeurs ?

Pour R1 :

R1 est directement connecté à 192.168.10.0/24 (eth0), 192.168.20.0/24 (eth1) et 192.168.1.0/24 (eth2)

Pour R2 :

R2 est directement connecté à 192.168.30.0/24 (eth0), 192.168.2.0/24 (eth2) et 192.168.1.0/24 (eth1)

Pour R3 :
R3 est directement connecté à 192.168.40.0/24 (eth0) et 192.168.2.0/24 (eth1).

4. Vous devez déclarer chaque réseau relié directement au routeur avec la commande network suivie de l'adresse du réseau (en notation CIDR). Effectuez cette opération sur le routeur R1 uniquement.

Pour R1 :

```
R1# configure terminal
R1(config)#
R1(config)# router rip
R1(config-router)#
R1(config-router)# network 192.168.10.0/24
R1(config-router)#
R1(config-router)# network 192.168.20.0/24
R1(config-router)#
R1(config-router)# network 192.168.1.0/24
R1(config-router)#
R1(config-router)#
```

5. Sortez du mode configuration avec Ctrl+D jusqu'à avoir juste l'invite de commande administrateur de Quagga : R1#

```
R1(config-router)# network 192.168.10.0/24
R1(config-router)#
R1(config-router)# network 192.168.20.0/24
R1(config-router)#
R1(config-router)# network 192.168.1.0/24
R1(config-router)#
R1(config-router)#
R1(config)#
R1#
```

6. Avec la commande show ip rip, affichez les réseaux connus de R1. Lesquels voit-on ? Ici encore, vous allez voir le réseau interne de Marionnet 172.23.0.0/16 : ne vous en préoccupez toujours pas.

```
R1# show ip rip
Codes: R - RIP, C - connected, S - Static, O - OSPF, B - BGP
Sub-codes:
      (n) - normal, (s) - static, (d) - default, (r) - redistribute,
      (i) - interface

     Network         Next Hop         Metric From            Tag Time
C(i) 192.168.1.0/24  0.0.0.0               1 self              0
C(i) 192.168.10.0/24 0.0.0.0               1 self              0
C(i) 192.168.20.0/24 0.0.0.0               1 self              0
R1#
```

Les réseaux connus de R1 sont :
- 192.168.1.0/24
- 192.168.10.0/24
- 192.168.20.0/24

7. Effectuez la même manipulation sur les deux autres routeurs. Une fois le routeur R2 configuré, attendez 30 secondes et tapez à nouveau la commande show ip rip sur R1. Que constatez-vous ?

Sur R2 :

```
R2# configure terminal
R2(config)#
R2(config)# router rip
R2(config-router)#
R2(config-router)# network 192.168.30.0/24
R2(config-router)#
R2(config-router)# network 192.168.2.0/24
R2(config-router)#
R2(config-router)# network 192.168.1.0/24
R2(config-router)#
R2(config-router)#
```

Les routes sur R1 :

```
R1# show ip rip
Codes: R - RIP, C - connected, S - Static, O - OSPF, B - BGP
Sub-codes:
      (n) - normal, (s) - static, (d) - default, (r) - redistribute,
      (i) - interface

     Network         Next Hop         Metric From            Tag Time
C(i) 192.168.1.0/24  0.0.0.0               1 self              0
R(n) 192.168.2.0/24  192.168.1.2           2 192.168.1.2       0 02:43
C(i) 192.168.10.0/24 0.0.0.0               1 self              0
C(i) 192.168.20.0/24 0.0.0.0               1 self              0
R(n) 192.168.30.0/24 192.168.1.2           2 192.168.1.2       0 02:43
R1#
```

On constate que les routes que R2 connaît R1 les connaît aussi et sait le chemin à faire.

8. Configurez les trois routeurs et assurez-vous avec show ip rip que tous les routeurs voient bien tous les sous-réseaux

Sur R3 :

```
R3# configure terminal
R3(config)#
R3(config)# router rip
R3(config-router)#
R3(config-router)# network 192.168.40.0/24
R3(config-router)#
R3(config-router)# network 192.168.2.0/24
R3(config-router)#
R3(config-router)#
```

Les routes sur R1 :

```
R1# show ip rip
Codes: R - RIP, C - connected, S - Static, O - OSPF, B - BGP
Sub-codes:
      (n) - normal, (s) - static, (d) - default, (r) - redistribute,
      (i) - interface

     Network            Next Hop         Metric From            Tag Time
C(i) 192.168.1.0/24     0.0.0.0               1 self              0
R(n) 192.168.2.0/24     192.168.1.2           2 192.168.1.2       0 02:41
C(i) 192.168.10.0/24    0.0.0.0               1 self              0
C(i) 192.168.20.0/24    0.0.0.0               1 self              0
R(n) 192.168.30.0/24    192.168.1.2           2 192.168.1.2       0 02:41
R(n) 192.168.40.0/24    192.168.1.2           3 192.168.1.2       0 02:41
R1#
```

Il s’agit de la même chose les routes de R3 sont aussi connu par R1.

R2 :

```
R2# show ip rip
Codes: R - RIP, C - connected, S - Static, O - OSPF, B - BGP
Sub-codes:
      (n) - normal, (s) - static, (d) - default, (r) - redistribute,
      (i) - interface

     Network            Next Hop         Metric From            Tag Time
C(i) 192.168.1.0/24     0.0.0.0               1 self              0
C(i) 192.168.2.0/24     0.0.0.0               1 self              0
R(n) 192.168.10.0/24    192.168.1.1           2 192.168.1.1       0 02:43
R(n) 192.168.20.0/24    192.168.1.1           2 192.168.1.1       0 02:43
C(i) 192.168.30.0/24    0.0.0.0               1 self              0
R(n) 192.168.40.0/24    192.168.2.2           2 192.168.2.2       0 02:35
R2#
```

R3 :

```
R3# show ip rip
Codes: R - RIP, C - connected, S - Static, O - OSPF, B - BGP
Sub-codes:
      (n) - normal, (s) - static, (d) - default, (r) - redistribute,
      (i) - interface

     Network            Next Hop         Metric From            Tag Time
R(n) 192.168.1.0/24     192.168.2.1           2 192.168.2.1       0 02:39
C(i) 192.168.2.0/24     0.0.0.0               1 self              0
R(n) 192.168.10.0/24    192.168.2.1           3 192.168.2.1       0 02:39
R(n) 192.168.20.0/24    192.168.2.1           3 192.168.2.1       0 02:39
R(n) 192.168.30.0/24    192.168.2.1           2 192.168.2.1       0 02:39
C(i) 192.168.40.0/24    0.0.0.0               1 self              0
R3#
```

9. Sortez de l interface de configuration de Quagga et affichez la table de routage de chaque routeur. Que constatez-vous ?

Pour R1 :

```
[0 root@R1 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     0.0.0.0         255.255.255.0   U     0      0        0 eth2
192.168.2.0     192.168.1.2     255.255.255.0   UG    2      0        0 eth2
192.168.10.0    0.0.0.0         255.255.255.0   U     0      0        0 eth0
192.168.20.0    0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.30.0    192.168.1.2     255.255.255.0   UG    2      0        0 eth2
192.168.40.0    192.168.1.2     255.255.255.0   UG    3      0        0 eth2
[0 root@R1 ~]$
```

Pour R2 :

```
[0 root@R2 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.2.0     0.0.0.0         255.255.255.0   U     0      0        0 eth2
192.168.10.0    192.168.1.1     255.255.255.0   UG    2      0        0 eth1
192.168.20.0    192.168.1.1     255.255.255.0   UG    2      0        0 eth1
192.168.30.0    0.0.0.0         255.255.255.0   U     0      0        0 eth0
192.168.40.0    192.168.2.2     255.255.255.0   UG    2      0        0 eth2
[0 root@R2 ~]$
```

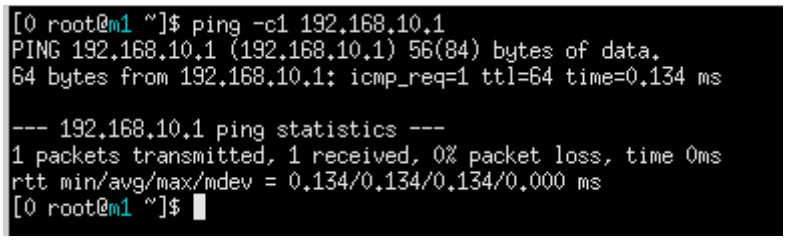
Pour R3 :

```
[0 root@R3 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     192.168.2.1     255.255.255.0   UG    2      0        0 eth1
192.168.2.0     0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.10.0    192.168.2.1     255.255.255.0   UG    3      0        0 eth1
192.168.20.0    192.168.2.1     255.255.255.0   UG    3      0        0 eth1
192.168.30.0    192.168.2.1     255.255.255.0   UG    2      0        0 eth1
192.168.40.0    0.0.0.0         255.255.255.0   U     0      0        0 eth0
[0 root@R3 ~]$
```

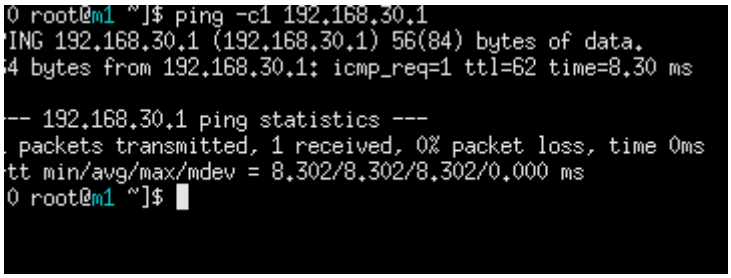
Les routes ont bien été attribué dynamiquement grâce au protocole RIP. Chaque routeur connaît tous les réseaux et sait où passer pour atteindre chaque réseau.

10. Assurez-vous avec ping que les machines peuvent se contacter entre elles.

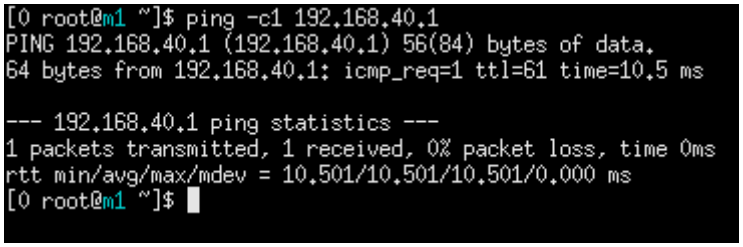
M1 vers m2 :



M1 vers m3 :



m1 vers m4 :



J’ai essayé pour m1 et les autres machines. Dans ces screens m1 peut contacter toutes les autres machines. (J’ai essayé avec les autres machines tout fonctionne je passe donc à la suite du TP)

Exo 4 : Ajout d'un routeur à chaud

1. Configurez Zebra et RIP sur R4 en ajoutant dans RIP les deux nouveaux sous-réseaux qui le relient à R1 et à R3.

Avant ça, nous avons rajouter le routeur dans le réseau, je configure ensuite les adresses de ses interfaces :

Je passe ensuite à la configuration Zebra et RIP.

Pour Zebra :

```
!=== EXAMPLES (tip: uncomment and adapt) ===
!interface eth1
!  ip   address 10.10.0.5/16
!  ipv6 address 2001:db9::5/64
!exit
!ip   route 11.11.11.0/24 10.10.255.254
!ipv6 route 2001:db8::/64 2001:db9::ff
!---

hostname R4
```

Pour RIP :

```
!=== EXAMPLES (tip: uncomment and adapt) ===
!
!  network 10.0.0.0/8
!  network eth0
!

hostname R4
```

On oublie pas de redémarrer les services :

```
[0 root@R4 ~]$ nano /etc/quagga/zebra.conf
[0 root@R4 ~]$ /etc/init.d/quagga stop
[1 root@R4 ~]$ /etc/init.d/quagga start
[0 root@R4 ~]$ nano /etc/quagga/ripd.conf
[0 root@R4 ~]$ /etc/init.d/quagga stop
[1 root@R4 ~]$ /etc/init.d/quagga start
[0 root@R4 ~]$
```

Voici les réseaux auxquelles R4 à accès directement :

```
R4# show ip route
Codes: K - kernel route, C - connected, S - static, R - RIP,
       O - OSPF, I - IS-IS, B - BGP, A - Babel,
       > - selected route, * - FIB route

C>* 127.0.0.0/8 is directly connected, lo
C>* 192.168.3.0/24 is directly connected, eth0
C>* 192.168.4.0/24 is directly connected, eth1
R4#
R4#
```

On configure le RIP :

```
R4# configure terminal
R4(config)#
R4(config)# router rip
R4(config-router)#
R4(config-router)# network 192.168.3.0/24
R4(config-router)#
R4(config-router)# network 192.168.4.0/24
R4(config-router)#
R4(config-router)#
```

2. Attendez quelques minutes (2 ou 3 suffiront) et affichez la table de routage de tous les routeurs. Que constatez-vous ? Pourquoi ?

Pour R1 :

```
[0 root@R1 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     0.0.0.0         255.255.255.0   U     0      0        0 eth2
192.168.2.0     192.168.1.2     255.255.255.0   UG    2      0        0 eth2
192.168.10.0    0.0.0.0         255.255.255.0   U     0      0        0 eth0
192.168.20.0    0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.30.0    192.168.1.2     255.255.255.0   UG    2      0        0 eth2
192.168.40.0    192.168.1.2     255.255.255.0   UG    3      0        0 eth2
[0 root@R1 ~]$
```

Pour R2 :

```
[0 root@R2 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.2.0     0.0.0.0         255.255.255.0   U     0      0        0 eth2
192.168.10.0    192.168.1.1     255.255.255.0   UG    2      0        0 eth1
192.168.20.0    192.168.1.1     255.255.255.0   UG    2      0        0 eth1
192.168.30.0    0.0.0.0         255.255.255.0   U     0      0        0 eth0
192.168.40.0    192.168.2.2     255.255.255.0   UG    2      0        0 eth2
[0 root@R2 ~]$
```

Pour R3 :

```
[0 root@R3 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     192.168.2.1     255.255.255.0   UG    2      0        0 eth1
192.168.2.0     0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.10.0    192.168.2.1     255.255.255.0   UG    3      0        0 eth1
192.168.20.0    192.168.2.1     255.255.255.0   UG    3      0        0 eth1
192.168.30.0    192.168.2.1     255.255.255.0   UG    2      0        0 eth1
192.168.40.0    0.0.0.0         255.255.255.0   U     0      0        0 eth0
[0 root@R3 ~]$
```

Pour R4 :

```
[0 root@R4 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.3.0     0.0.0.0         255.255.255.0   U     0      0        0 eth0
192.168.4.0     0.0.0.0         255.255.255.0   U     0      0        0 eth1
[0 root@R4 ~]$
```

Les routes de R4 ne sont pas affichées sur les tables de routage des autres routeurs car R1 et R3 n'ont pas encore déclaré ces réseaux dans la configuration RIP.

3. Sur R1 et R3 (les deux routeurs reliés aux sous-réseaux qui connectent R4), ajoutez les nouveaux sous-réseaux dans les réseaux diffusés par RIP.

Sur R3 :

```
R3(config-router)# network 192.168.3.0/24
R3(config-router)#
R3(config-router)# network 192.168.4.0/24
R3(config-router)#
R3(config-router)#
```

Sur R1 :

(Je les avais déjà rajouté je ne peux pas avoir le screen d'attribution)

```
R1(config)# router rip
R1(config-router)#
R1(config-router)# network 192.168.3.0
% [RIP] Unknown command: network 192.168.3.0
R1(config-router)#
R1(config-router)# network 192.168.3.0/24
There is a same network configuration 192.168.3.0/24
R1(config-router)#
R1(config-router)# network 192.168.4.0/24
There is a same network configuration 192.168.4.0/24
R1(config-router)#
R1(config-router)#
```

4. Affichez à nouveau les tables de routage des quatre routeurs. Que constatez-vous ? Pourquoi ?

Pour R1 :

```
[0 root@R1 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     0.0.0.0         255.255.255.0   U     0      0        0 eth2
192.168.2.0     192.168.1.2     255.255.255.0   UG    2      0        0 eth2
192.168.3.0     0.0.0.0         255.255.255.0   U     0      0        0 eth3
192.168.4.0     192.168.3.254   255.255.255.0   UG    2      0        0 eth3
192.168.10.0    0.0.0.0         255.255.255.0   U     0      0        0 eth0
192.168.20.0    0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.30.0    192.168.1.2     255.255.255.0   UG    2      0        0 eth2
192.168.40.0    192.168.1.2     255.255.255.0   UG    3      0        0 eth2
[0 root@R1 ~]$
```

Pour R2 :

```
[0 root@R2 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.2.0     0.0.0.0         255.255.255.0   U     0      0        0 eth2
192.168.3.0     192.168.1.1     255.255.255.0   UG    2      0        0 eth1
192.168.4.0     192.168.2.2     255.255.255.0   UG    2      0        0 eth2
192.168.10.0    192.168.1.1     255.255.255.0   UG    2      0        0 eth1
192.168.20.0    192.168.1.1     255.255.255.0   UG    2      0        0 eth1
192.168.30.0    0.0.0.0         255.255.255.0   U     0      0        0 eth0
192.168.40.0    192.168.2.2     255.255.255.0   UG    2      0        0 eth2
[0 root@R2 ~]$
```

Pour R3 :

```
[0 root@R3 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     192.168.2.1     255.255.255.0   UG    2      0        0 eth1
192.168.2.0     0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.3.0     192.168.4.254   255.255.255.0   UG    2      0        0 eth2
192.168.4.0     0.0.0.0         255.255.255.0   U     0      0        0 eth2
192.168.10.0    192.168.2.1     255.255.255.0   UG    3      0        0 eth1
192.168.20.0    192.168.2.1     255.255.255.0   UG    3      0        0 eth1
192.168.30.0    192.168.2.1     255.255.255.0   UG    2      0        0 eth1
192.168.40.0    0.0.0.0         255.255.255.0   U     0      0        0 eth0
[0 root@R3 ~]$
```

Pour R4 :

```
[0 root@R4 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     192.168.3.253   255.255.255.0   UG    2      0        0 eth0
192.168.2.0     192.168.4.253   255.255.255.0   UG    2      0        0 eth1
192.168.3.0     0.0.0.0         255.255.255.0   U     0      0        0 eth0
192.168.4.0     0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.10.0    192.168.3.253   255.255.255.0   UG    2      0        0 eth0
192.168.20.0    192.168.3.253   255.255.255.0   UG    2      0        0 eth0
192.168.30.0    192.168.4.253   255.255.255.0   UG    3      0        0 eth1
192.168.40.0    192.168.4.253   255.255.255.0   UG    2      0        0 eth1
[0 root@R4 ~]$
```

Les nouvelles routes apparaissent car la configuration RIP a bien été effectué sur les routeurs R1 ET r3.

5. Dans les tables de routage, vous voyez deux colonnes que vous n’avez pas vues jusqu’à maintenant avec le routage statique : il s’agit de “Metric”et “Ref”. À votre avis, à quoi correspond la colonne “metric” ?

La colonne métrique correspond au nombre de routeurs qu’il faut pour atteindre un certain réseau.

Exo 5 : Tolérance aux pannes et résilience

1. R3 dispose d adresses IP sur deux réseaux entre routeurs : une adresse sur le réseau entre R2 et R3 et une adresse sur le réseau entre R4 et R3. Depuis R1, effectuez un traceroute vers R3 en utilisant chacune de ces deux adresses. Que constatez-vous ?

```
[0 root@R1 ~]$ traceroute 192.168.4.253
traceroute to 192.168.4.253 (192.168.4.253), 30 hops max, 38 byte packets
 1  192.168.3.254 (192.168.3.254)  2.472 ms  1.150 ms  0.927 ms
 2  192.168.4.253 (192.168.4.253)  3.106 ms  1.497 ms  1.507 ms
[0 root@R1 ~]$ traceroute 192.168.2.2
traceroute to 192.168.2.2 (192.168.2.2), 30 hops max, 38 byte packets
 1  192.168.1.2 (192.168.1.2)  1.860 ms  1.493 ms  0.792 ms
 2  192.168.2.2 (192.168.2.2)  1.993 ms  1.077 ms  0.990 ms
[0 root@R1 ~]$
```

On constate que les 2 chemins fonctionnent avant la panne en simultané.

2. Débranchez R2 du réseau. Vous pouvez le faire de façon “pas trop propre” : commencez par débrancher les câbles qui le relient à R1 et à R3, puis éteignez-le.
En exécutant la commande
route sur les routeurs restants, observez l’évolution des tables de routage de ceux-ci. Par exemple, vous pouvez exécuter la boucle suivante qui effectue un appel à route toutes les 20 secondes :

Quelles modifications sont faites dans les tables de routage ?

Pour R1 :

```
[0 root@R1 ~]$ for t in 'seq 1 10' ; do route ; sleep 10 ; done
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     0.0.0.0         255.255.255.0   U     0      0        0 eth2
192.168.2.0     192.168.3.254   255.255.255.0   UG    3      0        0 eth3
192.168.3.0     0.0.0.0         255.255.255.0   U     0      0        0 eth3
192.168.4.0     192.168.3.254   255.255.255.0   UG    2      0        0 eth3
192.168.10.0    0.0.0.0         255.255.255.0   U     0      0        0 eth0
192.168.20.0    0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.40.0    192.168.3.254   255.255.255.0   UG    3      0        0 eth3
[0 root@R1 ~]$
```

Pour R3 :

```
[0 root@R3 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     192.168.4.254   255.255.255.0   UG    3      0        0 eth2
192.168.2.0     0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.3.0     192.168.4.254   255.255.255.0   UG    2      0        0 eth2
192.168.4.0     0.0.0.0         255.255.255.0   U     0      0        0 eth2
192.168.10.0    192.168.4.254   255.255.255.0   UG    3      0        0 eth2
192.168.20.0    192.168.4.254   255.255.255.0   UG    3      0        0 eth2
192.168.40.0    0.0.0.0         255.255.255.0   U     0      0        0 eth0
[0 root@R3 ~]$
```

Pour R4 :

```
[0 root@R4 ~]$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.1.0     192.168.3.253   255.255.255.0   UG    2      0        0 eth0
192.168.2.0     192.168.4.253   255.255.255.0   UG    2      0        0 eth1
192.168.3.0     0.0.0.0         255.255.255.0   U     0      0        0 eth0
192.168.4.0     0.0.0.0         255.255.255.0   U     0      0        0 eth1
192.168.10.0    192.168.3.253   255.255.255.0   UG    2      0        0 eth0
192.168.20.0    192.168.3.253   255.255.255.0   UG    2      0        0 eth0
192.168.40.0    192.168.4.253   255.255.255.0   UG    2      0        0 eth1
[0 root@R4 ~]$
```

Les changements des route sont effectué automatiquement sur les routeurs R1 et R3 car ils ont changé le chemin pour atteindre les réseaux perdus.

3. Depuis R1, essayez à nouveau d'exécuter les deux traceroute vers R3. Que remarquez-vous ?

```
[0 root@R1 ~]$ traceroute 192.168.4.253
traceroute to 192.168.4.253 (192.168.4.253), 30 hops max, 38 byte packets
 1  192.168.3.254 (192.168.3.254)  2.472 ms  1.150 ms  0.927 ms
 2  192.168.4.253 (192.168.4.253)  3.106 ms  1.497 ms  1.507 ms
```

Le routeur R1 s'adapte tout seul de façon automatique.

Conclusion :

Voici une reformulation structurée en paragraphes, qui transforme tes notes de TP en une conclusion professionnelle tout en conservant l'analyse de tes erreurs techniques :

Ce TP a permis de valider la mise en œuvre du protocole RIP au sein d'une infrastructure multi-routeurs simulée sous Marionnet. Grâce à l'utilisation de la suite Quagga et de ses démons Zebra et ripd, nous avons pu observer la capacité des routeurs à échanger dynamiquement leurs tables de routage. Cette automatisation permet au réseau de s'adapter de manière autonome aux changements de topologie, éliminant ainsi le besoin d'interventions manuelles systématiques propres au routage statique.

La phase de configuration a toutefois révélé plusieurs points de vigilance techniques. Une difficulté majeure est apparue lors de l'édition des fichiers de configuration (zebra.conf et ripd.conf) : l'insertion des *hostnames* en fin de fichier plutôt qu'au début a empêché la bonne reconnaissance des paramètres par les démons sur R2 et R3, bloquant ainsi l'apprentissage des routes adjacentes. De plus, l'extension du réseau avec l'ajout d'un quatrième routeur (R4) a souligné l'importance de la cohérence de l'adressage. L'omission de la déclaration des nouveaux sous-réseaux (192.168.3.0 et 192.168.4.0) sur les routeurs R1 et R3 a initialement empêché la convergence globale des tables de routage.

En conclusion, les compétences acquises durant cette séance sont directement transposables en milieu professionnel, notamment pour l'administration de réseaux de petite à moyenne taille. La maîtrise des concepts fondamentaux de RIP — tels que la métrique par saut, les mécanismes de timers et la convergence automatique — constitue un socle indispensable pour appréhender des protocoles de routage plus complexes comme OSPF ou BGP, massivement déployés dans les infrastructures de grandes entreprises et d'opérateurs.